# Lab Manual 08 Signals

**Q:** How does the OS communicate to an application process?

**A:** Signals

## 44 WHAT IS SIGNAL

A notification that intimates a receiving process about occurrence of an event. With pipes we communicated data, with signal we can communicate control command.

Signals can be sent

1. By a process to another.
2. By kernel to other processes e.g. when a child process call exit() SIGCHLD is sent to the parent process by the kernel by default it is ignored.

The only information is:

- The number identifying the signal.
- Signal interrupts a process and forces it to handle the event immediately.

## 45 UNDERSTANDING SIGNAL

Signal: A notification of an event

1. Event gains attention of the OS.
2. OS stops the application process immediately, sending it a signal.
3. **Signal handler** executes to completion.
4. Application process resumes where it left off.

**EXAMPLE**
User types Ctrl-c

1. Event gains attention of OS
2. OS stops the application process immediately, sending it 2/SIGINT signal
3. Signal handler for 2/SIGINT signal executes to completion

4. Default signal handler for 2/SIGINT signal exits process

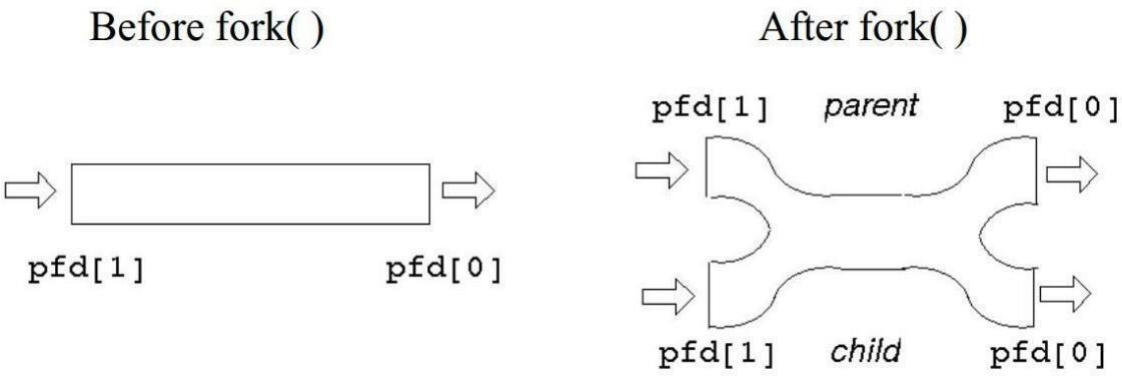



Figure 7: Signal

## 46 Actions Performed upon Receiving a Signal

1. Explicitly ignore the signal.
2. Execute the default action associated with the signal.
3. Catch the signal by invoking a corresponding signal-handler function.

## 47 Sending Signals via Keystrokes

1. Ctrl-c – 2/SIGINT signal Default handler exits process.
2. Ctrl-z – 20/SIGTSTP signal Default handler suspends process.
3. Ctrl- – 3/SIGQUIT signal Default handler exits process.

## 48 List of Signals and default action

kill -l can be used to view all the signals supported by your system.

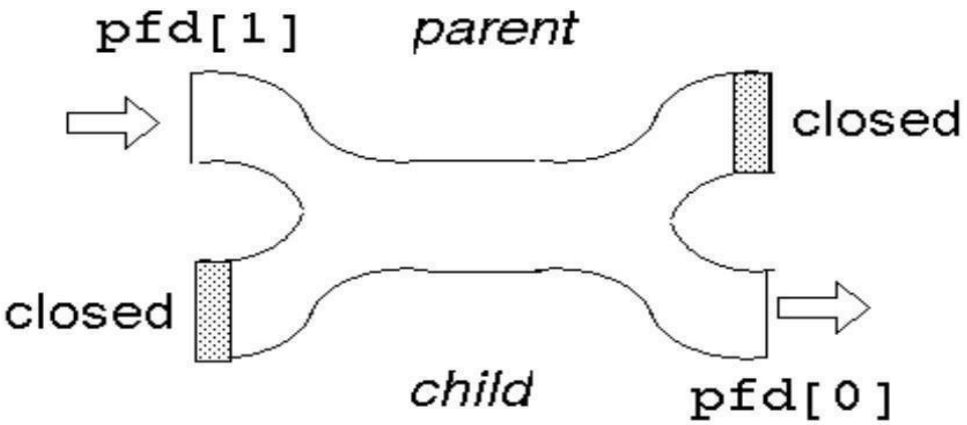



Figure 8: List of Signals and default action

## 49 THE SIGNAL() SYSTEM CALL

signal() system call is used to set signal handler for a signal type.

```
# include <signal . h>
 sighandler_t signal (int signum , sighandler_t handler ) ;
```

1. signal() sets the disposition of the signal signum to handler, which is one of the following:
   (a) SIG_IGN
   (b) SIG_DFL
   (c) The address of a programmer-defined function (a "signal handler")

2. signal() returns the previous value of the signal handler, or SIG_ERR on error.

## 50 SIGNAL HANDLER

- Corresponding to each signal is a signal handler.
- Called when a process receives a signal.
- The function is called "asynchronously".
- When the signal handler returns the process continues, as if it was never interrupted.

## 51 PRE-DEFINED SIGNAL HANDLERS

Pre-defined Signal Handlers are:

1. **SIG_IGN:**

   Causes the process to ignore the specified signal.
   *Example:* To ignore Ctrl-C completely – signal(SIGINT, SIG_IGN);

2. **SIG_DFL:**

   Causes the system to set the default signal handler for the given signal *Example:*
   signal(SIGTSTP, SIG_DFL);

**EXAMPLE**

```
# include        <iostream >
# include       < s t d l i b . h>
# include        <signal . h>
#   include <unistd . h> using
namespace std ;

int main ( ) { signal (SIGQUIT , SIG_IGN ) ; //ignoring CTRL backlash signal (SIGTSTP ,
               SIG_IGN ) ; //ignoring CTRL Z signal (SIGINT , SIG_IGN ) ; //ignoring CTRL C

               for ( int i =0;; i ++){ cout<<i <<" My pid is : "<<getpid ()<<endl ; cout<<"You can ' t close me using
                        conventional commands"<<endl ;
                        sleep ( 2 ) ;
               }
return 0;

}
```

## 52 USER DEFINED SIGNAL HANDLERS

1. User can Define a functions to be executed on receiving different signals.
2. Most of the signals can be caught or handled except SIGKILL and SIGSTOP.
3. Signal Handlers can not be called at any point except for the signal generation.

**EXAMPLE**

```
# include     <iostream >

# include     < s t d l i b . h>

# include     <signal. h>

# include     <unistd. h>

using namespace std ; void

myhandler ( int ) ;

int main ( ) { signal (SIGINT, myhandler ) ; //adding custom handler
                                                                    to CTRL-C
                        i =0;; i ++){
 cout<<i <<endl ;
                        //just        slowing down execution     so  can     observe  better
 sleep ( 2 ) ;
            }
return 0;
}

void myhandler ( int signal_number ) { signal (SIGINT, myhandler ) ; //resetting the
            signal
            cout<<"    Inside my custom handler and received signal "
                        <<signal_number<<endl ;
 }
```

## 56 THE KILL() SYSTEM CALL

The kill() system call can be used to send any signal to any process. If pid is positive, then signal sig is sent to pid.

```
int  kill ( pid_t pid ,  int  sig ) ;

-SIGKILL Pid
```

To send signal using terminal the syntax for kill command is:

**kill** [ signal or option ]

$ **kill** PID( s )

On success, zero is returned. On error, -1 is returned, and errno is set appropriately. To check if a process exists or not use kill() by sending signal#0 to that process.

```
I f ( kill ( pid ,0)==0)
                process       exist
else do not exist
```

**EXAMPLE 01**

```
# include          <iostream >
# include         < s t d l i b . h>
# include          <signal . h>

 # include <unistd .h>
> # include <sys / wait . h>
 using namespace std ;
int main ( ) {

            pid_t pid ; pid =
            fork ( ) ;
             if ( pid <0){ cout<<" Child process not created "<<endl ;
            }

            else if ( pid >0){ cout<<" I am parent with pid "<<getpid ( )
                                    <<"   and    child    pid    is     "<<pid<<endl ;
                        wait (NULL) ;
            }

            else { cout<<" I am child with pid "<<getpid ( )
                                    <<"     and my parent      pid  is          "<<getppid ( )
                                    <<endl ;
                        kill ( getppid ( ) , SIGKILL ) ;
                        cout<<"My parent is now "<<getppid ()<<endl ; sleep ( 1 ) ;
                           cout<<"My parent       is                now "<<getppid ()<<endl ;
            }

            return 0;
}
```

**EXAMPLE 02**

```
# include        <iostream >
# include       < s t d l i b . h>
# include        <signal . h>
# include <unistd . h> # include <sys / wait . h> using namespace std ;

void handler ( int signum ) { cout<<" Signal is received "
                        <<signum<<endl ;
            sleep ( 5 ) ;
            wait (NULL) ;
}

int main ( ) { int cpid , ppid ; signal (SIGUSR1 , handler ) ; signal (SIGCHLD, handler ) ;
            cpid = fork ( ) ; switch ( cpid ) { case −1: cout<<" Error "<<endl ; break ; case 0: ppid =getppid ( ) ;
                                    kill ( ppid , SIGUSR1 ) ;
                                  exit ( 0 ) ; break ;


                        default : while ( kill ( cpid , 0)==0) cout<<" waiting for child "<<endl ; cout<<"
                        Child
                                    terminated . Now I am terminating "<<endl ;


              }
                return   0;
  }
```

## 57 MORE SYSTEM CALLS

1. **PAUSE**

```
int     pause ( ) ;
```

Suspend the calling process until any signal is received.

**EXAMPLE**

```
# include      <iostream >

# include      < s t d l i b . h>

# include      <signal . h>

# include      <unistd . h>

# include      <sys / wait . h>
```

```
using     namespace     std ;

void myhandler ( int signum ) { cout<<" Processing a SIGINT
            signal
                        <<signum<<endl ;                    "
}

int
      main ( ) { signal (SIGINT , myhandler ) ; cout<<" Begi
            execution –touch CTRL                    –C to     continue     program"
                        <<endl ;
             for ( int      i =0; i <5; i ++){
                        cout<<" Value          of          i          is
                          "<<i <<endl ; pause ( ) ;
            }
            return 0;
}
```

2. **RAISE**

```
int    raise ( int       sig )
```

Sends the signal sig to the executing program. raise() actually uses kill() to send the signal to the executing program:

**EXAMPLE**

```
# include        <iostream >
# include       < s t d l i b . h>
# include        <signal . h>
# include <unistd . h> # include
<sys  /  wait  .  h>  using
namespace std ;

int main ( ) { cout<<"A suicidal program"<<endl ;
                raise      ( SIGKILL ) ;
               cout<<" Terminating . . . "<<endl ;
               return 0;
}
```

3. **ALARM**

```
int    alarm ( unsigned       int    sec )
```

- alarm - set an alarm clock for delivery of a signal.
- alarm() arranges for a SIGALRM signal to be delivered to the calling process in sec seconds.
- If sec is zero, any pending alarm is canceled.

## 58 WHAT IS SIGNALS SET?

A signal set is a collection of one or more signals just like we define a set in mathematics.

1. Need a data type to represent multiple signals.
2. The sigset_t data type is used to represent a signal set.

```
s i gs e t _t    myset
```

## 59 FUNCTIONS TO MANIPULATE SIGNAL SETS

Following are the functions to manipulate signal sets:

```
Sigemptyset ( si g s e t_ t ∗set )
```

- Empty a signal set
- e.g. sigemptyset(&mysignals);

```
S i g f i l l s e t ( s i g s et _ t ∗set )
```

- Initialize a signal set with all signals
- e.g. sigfillset(&mysignals);

```
Sigaddset ( s i gs e t _ t ∗set ,        int    signo )
```

- Add sig.no into the specified set
- e.g.sigaddset(&mysignals,SIGUSR1)

```
Sigdelset ( s i gs e t _t    ∗set ,     int    signo )
```

- Delete signo from the specified set
- e.g. sigdelset(&mysignals,SIGUSR1)

```
Sigismember ( s ig s e t_ t    ∗set ,     int    signo )
```

- Returns True if sig.no is the member of the set specified in sigset_t.
  - e.g. sigismember(&mysignals,SIGINT)

## 60 EXAMPLE: MANIPULATING SIGNAL SETS

```
# include      <s t d i o . h>

# include      < s t d l i b . h>

# include     <signal . h>

#  include    <errno  .  h>

int


       main ( ) { s i gs e t _t sigset ;
              // f i l l sigset with a l l available OS signals s i g f i l l s e t (& sigset
               ) ;
               /∗    Checking     for     membership∗/     if     (
               sigismember(& sigset , SIGINT ) ) printf ( "
               SIGINT" ) ;
               if ( sigismember(& sigset , SIGQUIT ) ) printf ( " SIGQUIT" )
                              ;
               if ( sigismember(& sigset , SIGUSR1 ) ) printf ( " SIGUSR1"
                             ) ;
               if  (  sigismember(&  sigset  ,  SIGALRM)  )  printf  (  "
                             SIGALRM\n" ) ;

               /∗ Deleting SIGUSR1 from sigset ∗/ sigdelset (&
               sigset , SIGUSR1 ) ; if ( sigismember(& sigset ,
               SIGUSR1 ) ) printf ( "SIGUSR1\n" ) ;
               else printf ( "SIGUSR1 is Now not member of

                            /∗ Empty     the       sigset ∗/
               sigemptyset(& sigset ) ;
               if ( sigismember(& sigset , SIGALRM) ) printf ( " SIGALRM"
                             ) ;
               else printf ( " Sigset      is            empty\n" ) ;                        sigset      \n" ) ;

                 return  0;
}
```

## 61 SIGPROCMASK FUNCTION

Each process has a signal mask that defines the set of signals currently blocked from delivery to that process. A process can:

1. Examine its signal mask

2. Change its signal mask

3. Perform both operations in one step by calling the following function:

```
int     sigprocmask ( int       how, const      s ig s e t_ t ∗set ,      s ig s e t_ t ∗ oset ) ;
```

**FUNCTION PARAMETERS**

1. **const sigset_t *set**
   New signal mask of sigset_t type

2. **sigset_t *oset**
   Old signal mask of sigset_t type

3. **int how**

(a) **SIG_BLOCK** The set of blocked signals is the union of the current set and the set argument.

(b) **SIG_UNBLOCK** The signals in set are removed from the current set of blocked signals. It is legal to attempt to unblock a signal which is not blocked.

(c) **SIG_SETMASK** The set of blocked signals is set to the argument set.

62 **EXAMPLE: SIGPROCMASK FUNCTION**

```
# include      <s t d i o . h>
# include      < s t d l i b . h>
# include       <signal . h>
# include <unistd . h> # include <sys / wait . h> int main ( )
{
            // new_mask set to          store      different signals s i gs e t _t  new_mask ;
               //     I n i t i l i z e          and Empty new_mask set sigemptyset(&new_mask ) ;
            // Adding SIGQUIT to set named as new_mask sigaddset(&new_mask , SIGQUIT ) ;
```

```
        // Adding SIGTSTP to set named as new_mask sigaddset(&new_mask , SIGTSTP ) ;
        // Adding SIGINT to set named as new_mask sigaddset(&new_mask , SIGINT ) ;

        /*
                    * Block    All the signal          present in new_mask set .
        */ sigprocmask (SIG_BLOCK,&new_mask ,NULL) ;
          int i =0;
        for ( i ; ; i ++){ printf ( "\n %d \n" , i ) ; printf ( "My PID is %d\n" , getpid ( ) ) ; printf ( "you can ' t close
                me using conventional commands\n" ) ; sleep ( 2 ) ;
           }
        return 0;
}
```

63 **SIGPENDING FUNCTION**

```
int  sigpending ( si g s et _ t    *set ) ;
```

The sigpending function returns the set of signals that are blocked from delivery and currently pending for the calling process. The set of signals is returned through the set argument.

**EXAMPLE**

```
# include    <s t d i o . h>
# include    < s t d l i b . h>
# include <signal . h> void sig_quit ( int signo ) {
printf ( " caught SIGQUIT\n" ) ;

            signal (SIGQUIT ,    SIG_DFL ) ;   //Reseting   the   signal  handler   for SIGQUIT
}
void      sig_term ( int signo ) { printf ( " caught
          SIGINT\n" ) ;
            signal (SIGINT ,    SIG_DFL ) ;   //Reseting      thesignal   handler   for SIGINT
}
int  main( void ) { s i gs e t _t newmask, oldmask , pendmask ;
          signal (SIGQUIT , sig_quit ) ; signal (SIGINT , sig_term
          ) ;

            //Block SIGQUIT and      save   current   signal   mask .
          sigemptyset(&newmask          )       ;
          sigaddset(&newmask,    SIGQUIT   )    ;
          sigaddset(&newmask, SIGINT ) ;

            // apply    "newmask    set "    as mask for process        and
```

```
        store      old mask      to      "oldmask      set "
        sigprocmask (SIG_BLOCK, &newmask, &oldmask ) ;
        printf ( " Sleep for 10 sec\n" ) ; sleep ( 1 0 ) ;

        // SIGQUIT && SIGINT here will remain pending sigpending(&pendmask ) ;
        if ( sigismember(&pendmask , SIGQUIT) && sigismember(&pendmask , SIGINT ) ) printf ( "\nSIGQUIT
                && SIGINT pending\n" ) ;
        //  Restore signal mask which unblocks SIGQUIT && SIGINT . //and This will unmask the
        process and deliver a l l pending sig sigprocmask (SIG_SETMASK, &oldmask , NULL) ; printf (
        "SIGQUIT && SIGINT unblocked\n" ) ;

        while ( 1 ) { }
        exit ( 0 ) ;
          /* SIGQUIT here        will      terminate      with      core      f i l e */
}
```

## 64 SIGSUSPEND FUNCTION

Suspend the process untill any signal is received except sigmask.

```
int     sigsuspend ( const        si g s e t_ t     *sigmask ) ;
```

1. The signal mask of the process is set to the value pointed to by sigmask.

2. Then the process is suspended until a signal is caught or until a signal occurs that terminates the process.

3. If a signal is caught and if the signal handler returns, then sigsuspend returns, and the signal mask of the process is set to its value before the call to sigsuspend.

**EXAMPLE**

```
# include      <s t d i o . h>
# include      < s t d l i b . h>
# include       <signal . h>

void       sig_int ( int       signo ) {
              printf ( "\nin        sig_int :\n       " ) ;
}

void sig_suspend ( int signo ) { printf ( "\nin sig handler with %d number:\n " , signo )
; }

int main ( ) { s i gs e t _t newmask, oldmask , waitmask ; printf ( "program s t a r t :
          with pid %d \n" , getpid ( ) ) ; signal (SIGINT , sig_int ) ;
```

```
        signal (SIGALRM, sig_suspend ) ; signal
        (SIGUSR1     ,     sig_suspend     )     ;
        sigemptyset(&waitmask          )          ;
        sigaddset(&waitmask   ,   SIGUSR1   )   ;
        sigaddset(&waitmask    ,    SIGALRM)    ;
        sigemptyset(&newmask          )          ;
        sigaddset(&newmask, SIGINT ) ;
        /*
                        * Block      SIGINT and     save  current       signal  mask .
        */ sigprocmask (SIG_BLOCK, &newmask, &oldmask ) ;

        /*
         * C r i t i c a l    region    of  code .
        */ printf ( " in    c r i t i c a l          region :

                \n" ) ;

        /*
         * I t will Pause u n t i l l received any signal except SIGUSR1 & SIGALRM.
                          * newmask set will       be     suspended for that       period
        */ sigsuspend(&waitmask ) ;

          // Now onwards old mask "newmask" is applicable printf ( " after
        return from sigsuspend : \n" ) ; sleep ( 1 5 ) ;
        /*
                                          * unblocking newmask set .
        */ sigprocmask (SIG_UNBLOCK, &newmask, NULL) ;

        /*
              * And continue processing          . . .
        */ printf ( "program          exit :
                \n" ) ;
        exit ( 0 ) ;
}
```

of time